

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Semestrálna práca z predmetu

Algoritmy a údajové štruktúry 2

DOKUMENTÁCIA VYUŽITIA AI

Vypracoval: Juraj Nižnanský

Študijná skupina: 5ZIB11

Akademický rok: 2026/2027

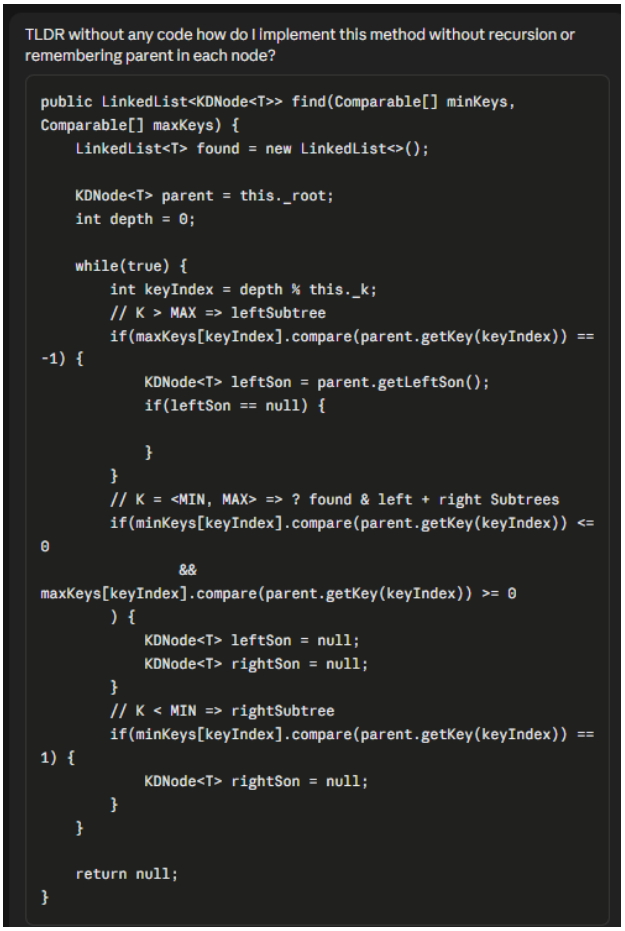
V Žiline dňa 8.11.2026

Obsah

1. Intervalové vyhľadávanie v KD-Strome 2
2. Načítanie slovakia_nodes.csv 3
3. Kontrola obsahu priečinka 4
4. Výber súboru na načítanie do aplikácie 5
5. Vyhľadávanie bodov POI podľa polomeru 6
6. Zanedbanie diakritiky pri porovnávaní názvov 7

1. Intervalové vyhľadávanie v KD-Strome

Pri implementácii intervalového vyhľadávania v KD-Strome bol využitý model **Claude Sonnet 5.5, medium** umelej inteligencie. AI nevygenerovala konkrétny kód, no poradila, akým spôsobom sa dá funkcia implementovať bez využitia rekurzie pomocou zásobníkov.



TLDR without any code how do I implement this method without recursion or remembering parent in each node?

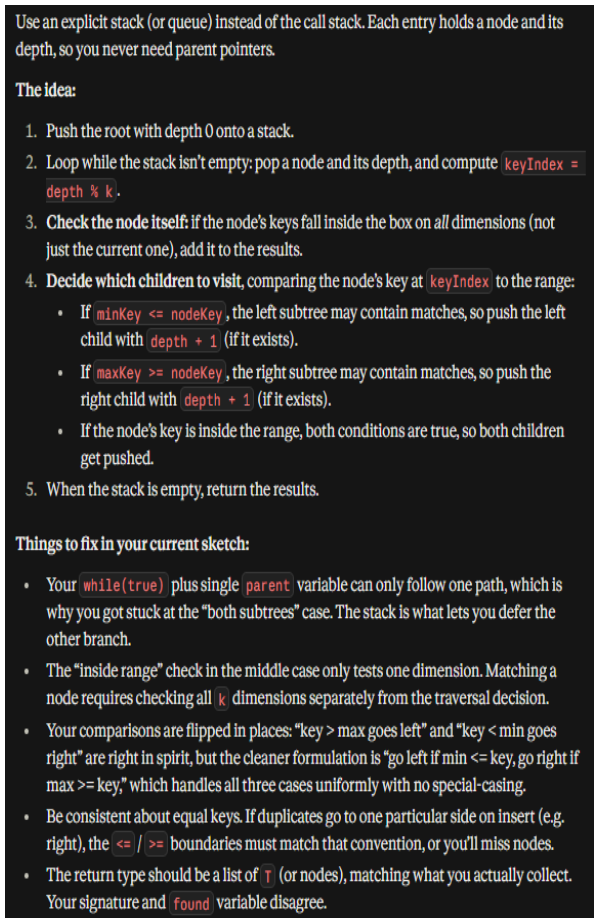
```
public LinkedList<KDNode<T>> find(Comparable[] minKeys,
Comparable[] maxKeys) {
    LinkedList<T> found = new LinkedList<>();

    KDNode<T> parent = this._root;
    int depth = 0;

    while(true) {
        int keyIndex = depth % this._k;
        // K > MAX => leftSubtree
        if(maxKeys[keyIndex].compare(parent.getKey(keyIndex)) ==
-1) {
            KDNode<T> leftSon = parent.getLeftSon();
            if(leftSon == null) {

            }
        }
        // K = <MIN, MAX> => ? found & left + right Subtrees
        if(minKeys[keyIndex].compare(parent.getKey(keyIndex)) <=
0
                &&
maxKeys[keyIndex].compare(parent.getKey(keyIndex)) >= 0
        ) {
            KDNode<T> leftSon = null;
            KDNode<T> rightSon = null;
        }
        // K < MIN => rightSubtree
        if(minKeys[keyIndex].compare(parent.getKey(keyIndex)) ==
1) {
            KDNode<T> rightSon = null;
        }
    }

    return null;
}
```



Use an explicit stack (or queue) instead of the call stack. Each entry holds a node and its depth, so you never need parent pointers.

The idea:

1. Push the root with depth 0 onto a stack.
2. Loop while the stack isn't empty: pop a node and its depth, and compute `keyIndex = depth % k`.
3. **Check the node itself:** if the node's keys fall inside the box on *all* dimensions (not just the current one), add it to the results.
4. **Decide which children to visit**, comparing the node's key at `keyIndex` to the range:
   - If `minKey <= nodeKey`, the left subtree may contain matches, so push the left child with `depth + 1` (if it exists).
   - If `maxKey >= nodeKey`, the right subtree may contain matches, so push the right child with `depth + 1` (if it exists).
   - If the node's key is inside the range, both conditions are true, so both children get pushed.
5. When the stack is empty, return the results.

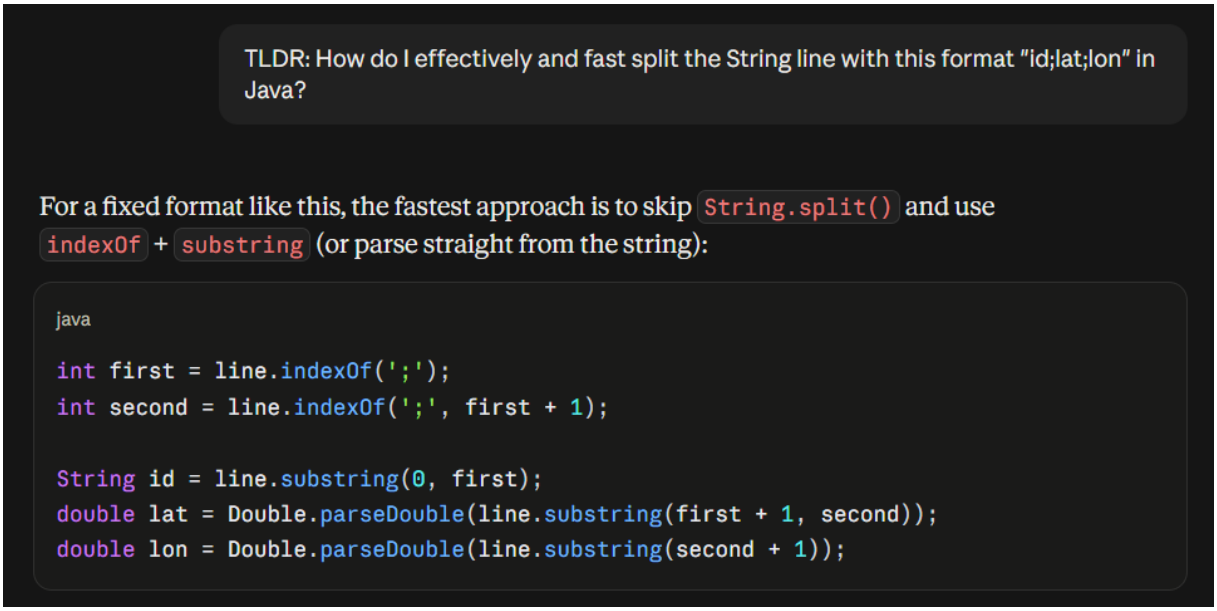
Things to fix in your current sketch:

- Your `while(true)` plus single `parent` variable can only follow one path, which is why you got stuck at the "both subtrees" case. The stack is what lets you defer the other branch.
- The "inside range" check in the middle case only tests one dimension. Matching a node requires checking all `k` dimensions separately from the traversal decision.
- Your comparisons are flipped in places: "key > max goes left" and "key < min goes right" are right in spirit, but the cleaner formulation is "go left if min <= key, go right if max >= key," which handles all three cases uniformly with no special-casing.
- Be consistent about equal keys. If duplicates go to one particular side on insert (e.g. right), the `<=` / `>=` boundaries must match that convention, or you'll miss nodes.
- The return type should be a list of `T` (or nodes), matching what you actually collect. Your signature and `found` variable disagree.

Link: https://claude.ai/share/dd611864-01cb-486f-a9d7-5ef0d08d16ca

2. Načítanie slovakia_nodes.csv

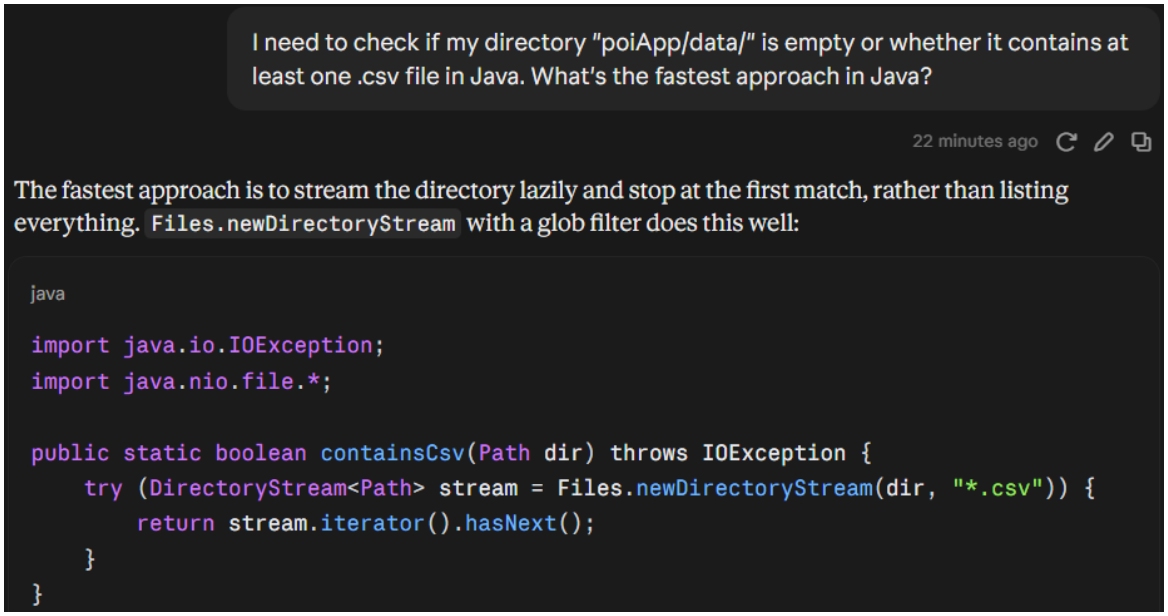
Pri implementácii načítavania CSV súboru bol využitý model **Claude Sonnet 5.5, medium** umelej inteligencie. AI vygenerovala kód na rýchle delenie String-u.



Link: https://claude.ai/share/cdc08eee-e656-42a2-abb1-1d27a38fe828

3. Kontrola obsahu priečinka

Pri implementácii kontroly priečinka (t.j. či obsahuje CSV súbor) bol využitý model **Claude Sonnet 5.5, medium** umelej inteligencie. AI vygenerovala kód, ktorý overí, či priečinok obsahuje aspoň jeden CSV súbor. Funkcia nakoniec nebola po refaktore použitá.



Link: https://claude.ai/share/a849b7d8-f27b-4e8d-825e-902d6219b17e

4. Výber súboru na načítanie do aplikácie

Pri implementácii načítania názvov súborov a pridania tlačidiel, vďaka ktorým sa načítajú korektné dáta bol využitý model **Claude Sonnet 5.5, medium** umelej inteligencie. AI vygenerovala kód, ktorý pre každý názov súboru pridá Consumer interface s metódou onSelect, ktorá sa volá v Presenteri.

Do môjho view potrebujem pridať skrolovateľný panel, ktorý bude obsahovať nasledovné:
|nazov_suboru_z_urceneho_priecinka.csv [Vybrať]|

Potrebujem v mojom presenteri zaregistrovať action listener na button vybrať, ktorý zoberie String fileToLoad, ktorý reprezentuje názov súboru a načíta dáta v metóde on FileSelected(String fileToLoad).

Ako to urobím čo najjednoduchšie?

```
public void setFiles(List<String> fileNames, Consumer<String> onSelect) {
    this.filesPanel.removeAll();

    for (String fileName : fileNames) {
        JPanel row = new JPanel(new BorderLayout());
        row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 35));

        JButton selectButton = new JButton("Vybrať");
        selectButton.addActionListener(e -> onSelect.accept(fileName));

        row.add(new JLabel(fileName), BorderLayout.CENTER);
        row.add(selectButton, BorderLayout.EAST);
        this.filesPanel.add(row);
    }

    this.filesPanel.revalidate();
    this.filesPanel.repaint();
}
```

```
private List<String> listCsvFiles() {
    File folder = new File("data"); // tvoj priečinok
    String[] names = folder.list((dir, name) -> name.endsWith(".csv"));
    return names == null ? List.of() : Arrays.asList(names);
}
```

Link: https://claude.ai/share/b012578b-c46d-4a99-a66d-c9c46d883d1a

5. Vyhľadávanie bodov POI podľa polomeru

Pri implementácii vyhľadávania bodov podľa polomeru bol využitý model **Cloude Sonnet 5.5, medium** umelej inteligencie. AI vygenerovala kód na výpočet tzv. Haversínovho vzorca, ktorý slúži na výpočet vzdialenosti medzi dvomi bodmi v prípade, že poznáme ich geografickú šírku a dĺžku v stupňoch. Vzorec zanedbáva sploštenie zeme a počíta s priemerným polomerom Zeme 6371 km.

I have got interval search implemented on my spatial tree structures basically like this:
find(double[] minKeys, double[] maxKeys) in Java. Now I need to find data from the tree when I know:

- latitude (degrees)
- longitude (degrees)
- radius (meters)

How do I retrieve rectangle on values with my find method and then filter out values that are not in the radius? TLDR

```
static final double EARTH_RADIUS_M = 6_371_000.0;

double latDelta = Math.toDegrees(radius / EARTH_RADIUS_M);
double lonDelta = Math.toDegrees(radius / (EARTH_RADIUS_M * Math.cos(Math.toRadians(la

double[] min = { lat - latDelta, lon - lonDelta };
double[] max = { lat + latDelta, lon + lonDelta };
```

```
static double haversine(double lat1, double lon1, double lat2, double lon2) {
    double dLat = Math.toRadians(lat2 - lat1);
    double dLon = Math.toRadians(lon2 - lon1);
    double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
             + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
             * Math.sin(dLon / 2) * Math.sin(dLon / 2);
    return 2 * EARTH_RADIUS_M * Math.asin(Math.sqrt(a));
}
```

```
boolean pole = latMax >= 90 || latMin <= -90;
latMin = Math.max(latMin, -90);
latMax = Math.min(latMax, 90);
if (pole) { lonMin = -180; lonMax = 180; }

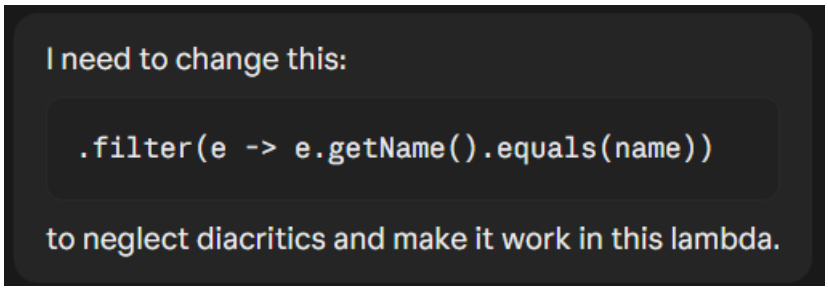
List<T> candidates = new ArrayList<>();

if (lonMin < -180) {           // Antimeridian, west overflow
    candidates.addAll(tree.find(new double[]{latMin, lonMin + 360}, new double[]{latMa
    candidates.addAll(tree.find(new double[]{latMin, -180},         new double[]{latMa
} else if (lonMax > 180) {     // Antimeridian, east overflow
    candidates.addAll(tree.find(new double[]{latMin, lonMin},  new double[]{latMax, 18
    candidates.addAll(tree.find(new double[]{latMin, -180},    new double[]{latMax, lo
} else {
    candidates.addAll(tree.find(new double[]{latMin, lonMin}, new double[]{latMax, lon
}
```

Link: https://claude.ai/share/3054ac39-d48e-475e-976a-fb424424e767

6. Zanedbanie diakritiky pri porovnávaní názvov

Pri implementácii zanedbávania diakritiky bol využitý model **Cloude Sonnet 5.5, medium** umelej inteligencie. AI vygenerovala kód, vďaka ktorému sa pri vyhľadávaní POI podľa názvu zanedbáva diakritika.




```java
import java.text.Normalizer;

private static String stripDiacritics(String s) {
    if (s == null) return null;
    return Normalizer.normalize(s, Normalizer.Form.NFD)
            .replaceAll("\\p{M}", "");
}
```

Link: https://claude.ai/share/69294d75-9e17-4e21-b68d-5dc681da6b43